Model Multipliers And Observer-Pole Adaptation

2026-04-01

Method Goal

This report groups the notebook families that adapt the internal prediction model rather than only the MPC cost. The main body covers TD3 and SAC methods that scale the augmented state-space matrices. The final section covers the observer-pole method, which keeps the plant model fixed but adapts the observer gain through pole placement.

Notebook Sources Covered

- `RL_assisted_MPC_matrices.ipynb`
- `RL_assisted_MPC_matrices_dist.ipynb`
- `RL_assisted_MPC_matrices_SAC.ipynb`
- `RL_assisted_MPC_matrices_dsit_SAC.ipynb`
- `RL_assisted_MPC_matrices_model_mismatch.ipynb`
- `RL_assisted_MPC_Poles.ipynb`

Matrix-Multiplier Method

State, Action, Reward

The TD3 and SAC matrix notebooks reuse the standard scaled RL state

$$s_t = \begin{bmatrix} \tilde{x}_t \\ \tilde{y}_{\mathrm{sp},t} \\ \tilde{u}_t^{\mathrm{dev}} \end{bmatrix}.$$

The action is continuous with dimension $1 + n_u$:

$$a_t \mapsto [\alpha_t, \delta_{t,1}, \ldots, \delta_{t,n_u}].$$

After remapping from $[-1, 1]$, the notebook uses these values to scale the augmented model matrices relative to immutable baselines:

$$A_t^{\text{now}} = \begin{bmatrix} \alpha_t A_{\text{phys}} & * \\ * & * \end{bmatrix}, \qquad B_t^{\text{now}} = B_{\text{base}} \operatorname{diag}(\delta_t)$$

in the implementation sense used by the notebooks:

$$A_{\text{now}}[: n_{\text{phys}}, : n_{\text{phys}}]* = \alpha_t, \qquad B_{\text{now}}[: n_{\text{phys}}, :]* = \delta_t.$$

Notebook-local matrix update logic

```
m_model = map_to_bounds(a_model_raw, model_low, model_high)
alpha = float(m_model[0])
delta = np.asarray(m_model[1:1 + n_inputs], float).reshape(-1)

A_now = A_base.copy()
B_now = B_base.copy()
n_phys = n_states - n_outputs
A_now[:n_phys, :n_phys] *= alpha
B_now[:n_phys, :] *= delta.reshape(1, -1)

MPC_obj.A = A_now
MPC_obj.B = B_now
```

TD3 Versus SAC

The TD3 notebooks use a deterministic actor with exploration noise and delayed policy updates. The SAC notebooks keep the same high-level control-loop shape but replace the actor with a Gaussian policy and entropy-regularized critic targets.

```
next_action, logp_next, mean_next = self.actor.sample(ns)
q_next = self.critic_target.combined_forward(ns, next_action, mode="min")
alpha = self.log_alpha.exp()
y = r + self.gamma * (1.0 - done) * (q_next - alpha * logp_next)
```

Mismatch Variant

The mismatch notebook keeps the same matrix-scaling mechanism but changes the operating conditions more aggressively. The purpose is to test whether online adaptation of (A, B) can keep the linear MPC layer useful when the plant moves away from the nominal identification point.

Observer-Pole Adaptation

Method Role

The pole notebook does not scale (A, B) directly. Instead, it lets a TD3 policy propose observer poles, sanitizes them, smooths them, and recomputes the observer gain L through pole placement.

Reusable observer-gain helper

```
def compute_observer_gain(A, C, desired_poles):
    obs_gain_calc = signal.place_poles(A.T, C.T, desired_poles,
                                       method='KNVO')
    L = np.squeeze(obs_gain_calc.gain_matrix).T
    observability_matrix = control.obsv(A, C)
    ...
    return L
```

Notebook-local pole update logic

```
if (i % pole_update_every) == 0:
    poles_prop = map_to_bounds(action, LOW_POLES, HIGH_POLES)
    poles_prop = sanitize_poles(poles_prop, LOW_POLES, HIGH_POLES)
    poles_cur = (1.0 - pole_smooth) * poles_cur + pole_smooth * poles_prop
    poles_cur = sanitize_poles(poles_cur, LOW_POLES, HIGH_POLES)
    try:
        L_new = compute_observer_gain(MPC_obj.A, MPC_obj.C, poles_cur)
        if np.all(np.isfinite(L_new)):
            L = L_new
    except Exception:
        pass
```

Core Algorithm Flow

1. Build the scaled RL state from the observer state and operating point.
2. Query either a TD3 or SAC agent for model multipliers, or a TD3 agent for observer poles.
3. Remap the action to physical coefficient bounds.
4. Update (A, B) or recompute L.
5. Solve MPC, apply the first move, step the plant, and roll the observer.
6. Store the transition and train the active RL agent after warm start.

Limitations, Caveats, And Open Questions

- The matrix and pole methods live mainly in notebook supervisors, not in a shared experiment module.
- The SAC notebooks change the learning rule but not the surrounding control loop, so careful comparison depends on keeping all notebook-level settings aligned.
- The repo also contains parallel TD3 implementations such as `agent_modified.py`; notebook imports must be checked before extending this family.

- Observer adaptation and matrix adaptation both depend strongly on the observer state, so stale helper modules should not be treated as the main execution path.